STUDENT

S-0006-GKL

TENTAMEN

250114 DT201G, DT202G

Kurskod	--
Bedömningsform	--
Starttid	14.01.2025 08:00
Sluttid	14.01.2025 13:00
Bedömningsfrist	--
PDF skapad	05.11.2025 13:14
Skapad av	Liselott Engelbrekts

i

Mittuniversitetet
MID SWEDEN UNIVERSITY

Tentamen

DT141G Operativsystemsteori
DT201G, DT202G Operativsystem

Jimmy Åhlander

Instruktioner

Tid	5 timmar
Resurser	Kalkylator, kurslitteratur, lexikon, egna anteckningar, penna och sudd. Läsplatta*.
Poäng	100
Krav för godkänt	För godkänt betyg på tentamen fordras betyg E eller högre.
Preliminära betygsgränser	A 90 % B 80 % C 70 % D 60 % E 50 %
Allmänt	Använd stycken, punktlistor och annan textformatering där det är lämpligt när du besvarar frågor med text. Om frågan kräver att du motiverar ditt svar, visar dina beräkningar eller ritar en figur erhålles i normalfallet inga poäng för endast svaret. Ibland är det emellertid endast en rekommendation och inte ett krav att svara på ett visst sätt. Exempel på krav: "*Rita en figur som styrker ditt svar*", "*Svara i bytes*", "*Visa dina beräkningar*". Exempel på rekommendationer: "*Rita en figur om du anser att det styrker ditt svar*", "*Tips: Skapa en tabell*". Tentan är anonymiserad. Skriv aldrig ditt namn, personnummer eller annan identifierande information i dina svar. Vid tekniska problem, kontakta alltid tentamensvakt. Stäng inte av eller starta om datorn utan uppsikt, om du inte redan lämnat in tentamen. *Det är inte tillåtet att använda eller ha tillgång till mobiltelefon, surfplatta eller liknande enheter under tentamen.

Endast läsplatta *utan tillgång till Internet*, av följande märke är tillåten: ReMarkable.

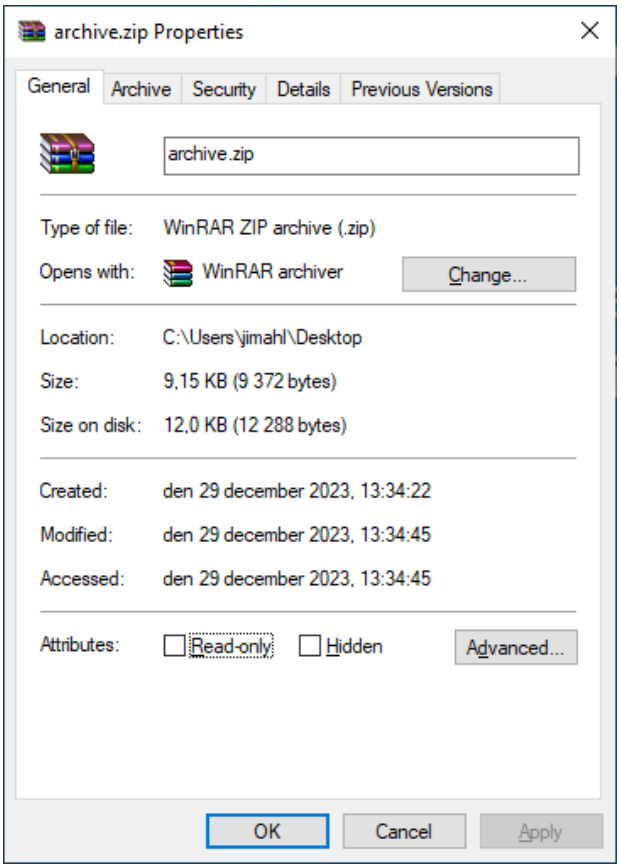
InsperaScan används ej för denna tentamen.

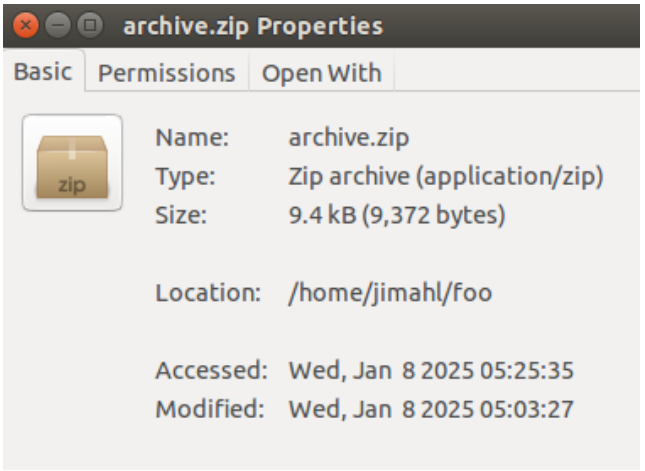
Mittuniversitetet
Institutionen för data- och elektroteknik (DET)

Jimmy Åhlander, 010 - 142 86 90

ℹ Filer hanteras på olika sätt i olika operativsystem. Generellt gäller emellertid att de har attribut och rättigheter samt att de lagras i någon form av katalogstruktur.

1(a) I följande bild visas egenskaperna för ett zip-arkiv i Windowsmiljö (NTFS). Egenskaperna för en kopia av samma arkiv i Linuxmiljö (ext4) visas i efterföljande bild.





Arkivets storlek anges på olika sätt och med olika värden, till synes inkonsekvent, mellan och inom systemen. Bland annat förekommer värdena 9,4, 9,15 och 12,0. Förklara varför samma arkivfil ger upphov till dessa tre olika värden för storleken och visa hur de beräknats fram.

Skriv in ditt svar här

De tre olika storleken på filen finns eftersom dom betyder egentligen olika saker. Storlekerna 9,4 och 9,15 är storleken av filen konverterat till kB. Dem är annorlunda eftersom windows och unix lika system har olika definitioner på vad en kb är. På unix lika system är en kb 1000 byte och på windows är dom 1024. Om vi tar storleken på filen i byte och konverterar dom med dessa definitioner får vi 9372/1000 blir 9.372, eller 9.4 när vi avrundar, och 9372/1024 blir 9.15. Den tredje storleken 12.0 är size on disk. Detta förekommer eftersom windows använder ett system som använder sig av chunks, detta betyder att en fil delas up i chunks av en viss förutbestämd storlek. Därför rundas storleken upp till storleken av alla chuncks som filen använder. I detta fal blir det 12 kb.

Ord: 137

Besvarad.

1(b) I följande bild ses delar av innehållet i katalogen där bland annat **`archive.zip`** återfinnes med hjälp av kommandot **`ls -lia`** i **`Bash`**. Besvara följande frågor utifrån denna information.

```
jimahl@ubuntu:~/foo$ ls -lia
total 20
1752751 dr-xr-x---  2 jimahl jimahl 4096 Jan  8 05:25 .
1609048 drwxr-xr-x 27 jimahl jimahl 4096 Jan  8 05:25 ..
1752752 -r-xrwxr--  3 jimahl jimahl 9372 Jan  8 05:03 archive.zip
1752753 lrwxrwxrwx  1 jimahl jimahl   11 Jan  8 05:20 bar -> archive.zip
```

Hur skulle filrättigheterna för archive.zip uttryckas på numeriskt (oktalt) format?

Svar: 574 .

Vilken grupp tillhör filen?

Svar: jimahl .

Hur många hårda länkar leder till samma inode som archive.zip (*från vad du kan se*)?

Svar: 3 stycken.

Hur många symboliska (mjuka) länkar leder till archive.zip (*från vad du kan se*)?

Svar: 1 stycken.

Vilket nummer har inoden för archive.zip?

Svar: 1752752 .

Rätt. 5 av 5 poäng.

1(c) Utgå ifrån tidigare presenterad information för att även besvara följande: Vilka av nedan angivna operationer är möjliga för användaren **`jimahl`**?

Välj ett eller flera alternativ:

- [x] Ändra filinnehållet i **`archive.zip`**.
- [x] Exekvera **`archive.zip`**, t.ex. som ett skalskript.
- [x] Skriva ut innehållet i **`archive.zip`**, t.ex. med **`cat`**.
- [x] Ändra namnet på **`archive.zip`**, t.ex. med **`mv`**.
- [x] Skriva ut innehållet i **`/home/jimahl/foo`**, t.ex. med **`ls`**.
- [ ] Skapa nya filer i **`/home/jimahl/foo`**, t.ex. med **`touch`**.
- [ ] Ta bort filer i **`/home/jimahl/foo`**, t.ex. med **`rm`**.
- [ ] Ta bort katalogen **`/home/jimahl/foo`**, t.ex. med **`rm -r`**.

Tips:

- Filnamn inklusive filändelser säger inget om filformatet.
- 4 av alternativen är korrekta.

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Delvis rätt. 1 av 4 poäng.

i Sidersättningsalgoritmer tillämpas i operativsystem som nyttjar virtuellt minne och demand paging. Givet tre fysiska ramar och sex logiska sidor, anta att processen P_1 använder sina logiska sidor i en sekvens angiven av en referenssträng där R representerar en läsoperation och W representerar en skrivoperation.

Processen P_1 har redan tre sidor allokerade i det fysiska minnet, se Tabell 1. P_1 behöver nu sidor enligt följande referenssträng: **`W3 W4 W2 R2 R1 W1 R4 R3 R4`**.

Tabell 1. Sidtabell för P_1.

Sida	Ram	SC bit	Smutsig/mod bit	Valid bit
0				0
1	0	0	0	1
2	1	1	0	1
3	2	0	1	1
4				0
5				0

Bilderna nedan representerar primärminnet uppdelat i sina ramar vid 10 olika tidpunkter: en för utgångsläget plus en för varje kommande sidreferens. Pekaren som leder till huvudet i den cirkulära offerkön representeras av symbolen **>**.

2(a) Visa hur sidorna i referenssträngen kommer att allokeras till olika ramar om algoritmen **Second Chance (SC)** tillämpas genom att fylla i vilken sida som är allokerad till vilken ram vid varje tidpunkt, det vill säga efter varje referens i referenssträngen.

Endast sidnumret ska anges i respektive fält.

1	1	4	4	4	4	4	4	4	4
2	2	2	2	2	2	2	2	3	3
>3	3	3	3	3	1	1	1	1	1

Tips:

- Skissa först med penna och papper.

Rätt. 5 av 5 poäng.

2(b) Hur många sidfel uppstår när SC tillämpas i det givna scenariot?

Antal sidfel: 3 .

Rätt. 1 av 1 poäng.

2(c) Vad är det minsta antalet sidfel som kan erhållas för det givna utgångsläget med den givna referenssträngen och tre ramar, oavsett algoritm?

Antal sidfel: 3 .

Fel. 0 av 2 poäng.

2(d) Vad är det största antalet sidfel som kan erhållas för det givna utgångsläget med den givna referenssträngen och tre ramar, oavsett algoritm?

Antal sidfel: 6 .

Rätt. 2 av 2 poäng.

ℹ Anta att du har ett operativsystem med arkitekturen **x86-64**, som använder paging för att allokera primärminnet. Sidstorleken är $4\ KiB$. Den effektiva åtkomsttiden till primärminnet har i detta system uppmätts till $80\ ns$.

3(a) Trots att `x86-64` stödjer 64-bitars virtuella adresser så är den fysiska adressrymden begränsad till 52 bitar i aktuell implementation. Hur mycket fysiskt minne kan i teorin användas i ett sådant system? *Svara i PiB.*

Svar: 4 PiB.

Tips:

- `1 PiB = 1024 TiB = 1024`^{2} `GiB`.

Rätt. 2 av 2 poäng.

3(b) När paging används kan minnet fragmenteras internt. Hur mycket minne kan en process maximalt förlora till intern fragmentering i det här systemet? *Svara i bytes*.

Svar: 4095 B.

Rätt. 2 av 2 poäng.

3(c) Processorn i systemet har precis genererat adressen `0xABCD1234FACE5678`. Vad blir sidnumret och förskjutningen för adressen?

Sidnummer: ABCD1234FACE5678/FFC .

Förskjutning: ABCD1234FACE5678%FFC .

Tips:

- Du kan svara i valfri talbas av följande: 2, 10, 16 utifrån den du tycker är enklast att arbeta med. Ange dock *inte* talbasen i svarsfälten.

Fel. 0 av 4 poäng.

3(d) Med paging och så pass stora minnesrymder som 64 bit måste processernas sidtabeller generellt lagras i primärminnet. Vad hade åtkomsttiden teoretiskt blivit om sammanhängande allokering använts istället för paging? *Ignorera att sidtabeller är hierarkiska i* `x86-64`. *Svara i ns*.

Svar: 40 ns.

Rätt. 2 av 2 poäng.

3(e) Processorer konstruerade enligt `x86-64` har en inbyggd TLB som en del av MMU:n. I det givna systemet används en TLB som har en träffsäkerhet på $95\ \%$ och en åtkomsttid på $17\ ns$.

Vi önskar nu sänka den effektiva åtkomsttiden till primärminnet genom att byta till en snabbare TLB. Vårt mål är en effektiv åtkomsttid om exakt $70\ ns$.

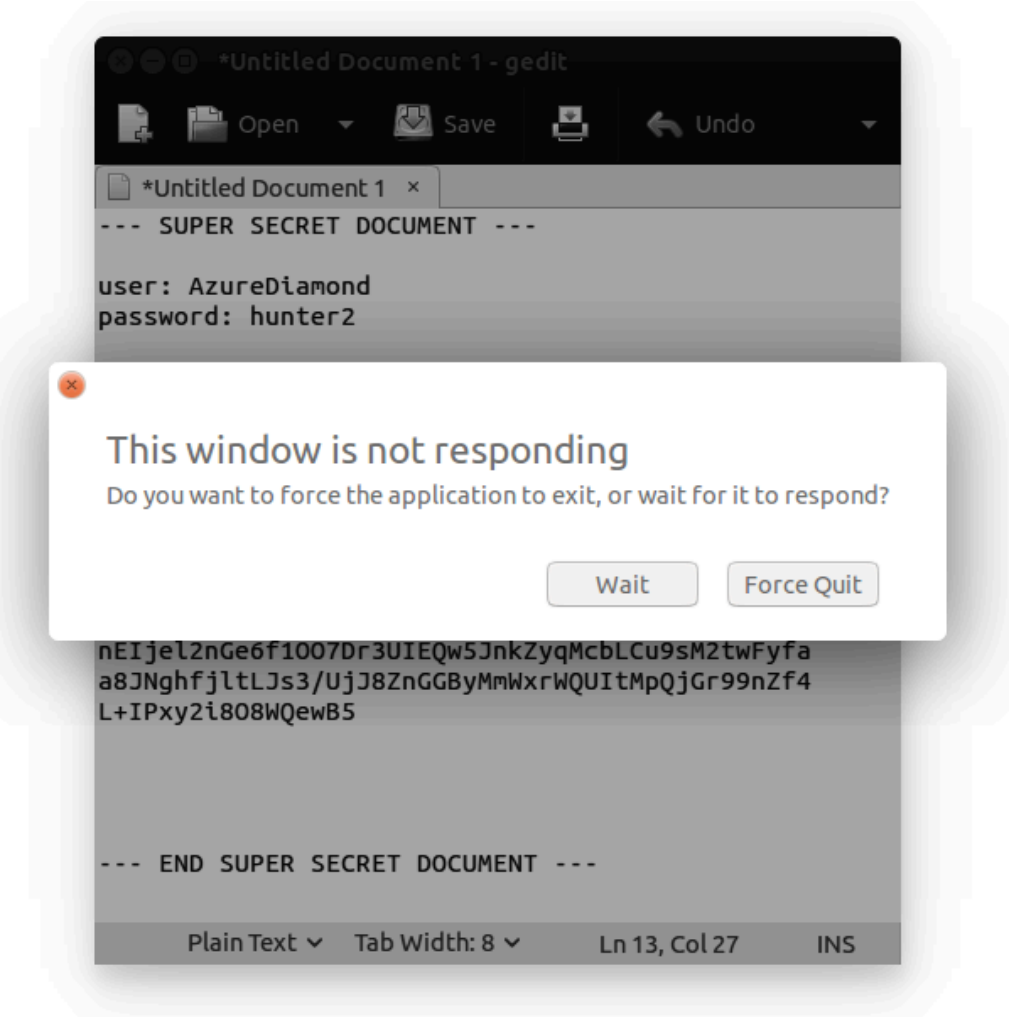
Vilken av följande buffertar uppfyller detta krav?

Välj ett alternativ:

- **`Buffert 1:`**
  $99\ \%$ träffsäkerhet, $17\ ns$ åtkomsttid
- **`Buffert 2:`**
  $95\ \%$ träffsäkerhet, $12\ ns$ åtkomsttid
- **`Buffert 3:`**
  $99\ \%$ träffsäkerhet, $12\ ns$ åtkomsttid
- **`Buffert 4:`**
  $95\ \%$ träffsäkerhet, $7\ ns$ åtkomsttid
- **`Buffert 5:`**
  $99\ \%$ träffsäkerhet, $7\ ns$ åtkomsttid
- Ingen buffert uppfyller kravet.

Rätt. 5 av 5 poäng.

Din tekniskt obegåvade, hypotetiska kusin som av någon spännande anledning börjat använda Ubuntu har hamnat i knipa. Kusinen, som inofficiellt går under pseudonymen AzureDiamond arbetade med ett viktigt dokument i gedit, till synes någon form av nyckel, när hen under en ouppmärksam sekund hamnade i följande situation:



Enligt utsaga ska kusins katt ha landat på tangentbordet och kommit åt några knappar. Vi kan varken bekräfta eller dementera uppgifterna rörande katten, men något har sannerligen hänt. I vilket fall är dokumentet osparat.

Du vill hjälpa din kusin att rädda dokumentet och har till din hjälp lyckats erhålla en skärmdump från Hishams table of processes (**`htop`**), se nedan. Besvara följande frågor med denna information i åtanke.

```
  PID USER      PRI  NI  VIRT   RES   SHR S CPU% MEM%   TIME+  Command
  770 root       20   0  344M  2952  2672 S  0.0  0.1  0:00.17 ├─ /usr/sbin/lightdm
 1026 root       20   0  224M  2832  2668 S  0.0  0.1  0:00.05 │  ├─ lightdm --session-child 12 19
 1260 kusin1     20   0 45776  2964  2116 S  0.0  0.1  0:00.70 │  │  ├─ /sbin/upstart --user
12730 kusin1     20   0  734M 49080 21960 S  1.0  2.4  3:46.14 │  │  │  ├─ /usr/lib/gnome-terminal-server
55688 kusin1     20   0     0     0     0 Z  0.0  0.0  0:00.07 │  │  │  │  ├─ bash
55646 kusin1     20   0 27528  5820  3112 S  0.0  0.3  0:00.08 │  │  │  │  ├─ bash
55730 kusin1     20   0  670M 40272 29332 T  0.0  2.0  0:06.70 │  │  │  │  │  └─ gedit
```

4(a) Vad är det underliggande problemet? Hur kan den här situationen ha uppstått?

Skriv in ditt svar här

Om vi kollar på htop så ser vi att gedits state är T. Detta betyder att programmet har hamnat i ett stopped lägge. Detta händer när en process får en vis signal som säger till den att pausa, sigstopVår kusin kan ha råkat stoppa processen genom att trycka in control z.

Ord: 51

Besvarad.

4(b) På vilket sätt kan den här situationen åtgärdas, rent praktiskt? *Förklara detaljerat tillvägagångssättet*.

Observera: Att kopiera dokumentet manuellt tecken för tecken är inte ett acceptabelt alternativ. Det inkluderar lösningar som bygger på skärmdumpar, OCR eller liknande. Ett enda misstag och nyckeln blir korrupt!

Skriv in ditt svar här

För att åtgärda situationen skulle jag först prova att använda fg i terminalen. Om det inte funkar skulle jag leta fram pid för processen i htop och använda den tillsammans med kommandot kill för att skicka SIGSTP till procesen.

Ord: 39

Besvarad.

i Din hypotetiska kusin har nu gett sig i kast med att lösa det klassiska *readers-writers problem*. Programkoden är skriven i Python och är inte komplett ännu varför din kusin nu söker din hjälp. Din kusins förklaring av koden lyder som följer:

I programmet samarbetar **två trådar** av klassen **`Reader`**. Trådarna delar en global variabel **`reader_count`** (initialiserad till 0) som bokför antalet trådar som befinner sig i den kritiska sektionen. Värdet ökas och sänks därmed med 1 i ingångs- respektive utgångssektionen av koden.

Ingångs- och utgångssektionen skyddas av två separata mutex-lås för att försäkra trådsäker åtkomst till **`reader_count`**. Varje tråd kör sin kod i **tre iterationer**. En slumpat lång sleep simulerar arbetet för den kod som ännu inte implementerats i ingångs- och utgångssektionerna. Koden kan ses nedan:

```
class Reader(Thread):
    def run(self):
        global reader_count

        for i in range(3): # 3 iterations

            # start entry section #
            entry_section_lock.acquire()
            count = reader_count
            time.sleep(random.uniform(0.00, 0.05))
            reader_count = count + 1
            entry_section_lock.release()
            # end entry section #

            # start critical section #
            # TODO: implement critical section
            # end critical section #

            # start exit section #
            exit_section_lock.acquire()
            count = reader_count
            time.sleep(random.uniform(0.00, 0.05))
            reader_count = count - 1
            exit_section_lock.release()
            # end exit section #
```

Du anar ugglor i mossen. Är koden verkligen synkroniserad och trådsäker?

5(a) Efter att de två trådarna exekverat klart bör värdet på **`reader_count`** vara 0. Vad är dock det högsta respektive lägsta värdet, oavsett sannolikhet, som **`reader_count`** kommer kunna erhålla efter en körning av programmet?

Högsta möjliga värde: 6 .

Lägsta möjliga värde: -5 .

Delvis rätt. 2 av 4 poäng.

5(b) Vilken av följande åtgärder är lämpligast för att åtgärda problemet?

Välj ett alternativ

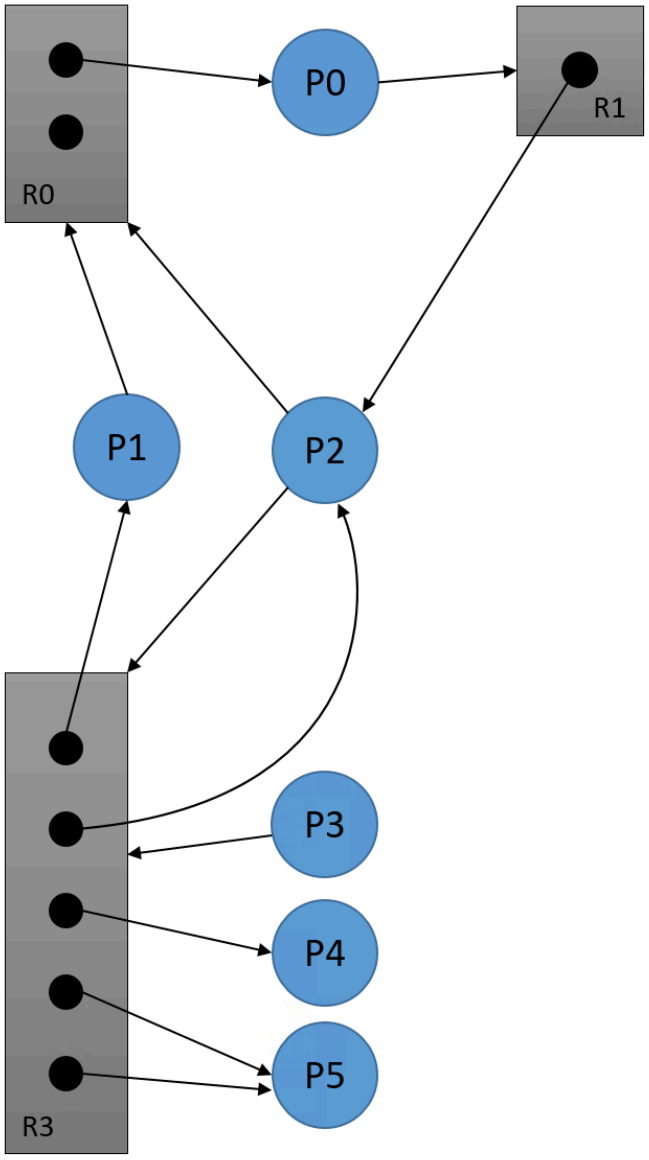
- Implementera ett nytt lås för den kritiska sektionen, t.ex. **`resource_lock`**
- Ta bort samtliga anrop till **`sleep`**.
- Ersätt **`entry_section_lock`** och **`exit_section_lock`** med ett gemensamt lås.
- Öka antalet iterationer varje tråd exekverar sin kod.
- Sänk antalet iterationer varje tråd exekverar sin kod.
- Ett nytt lås för att undvika svält för skrivare måste implementeras i ingångssektionen för läsaren, t.ex. **`read_try_lock`**.
- Ingen åtgärd är nödvändig. Värdet på **`reader_count`** kommer vara korrekt efter varje körning.

Rätt. 2 av 2 poäng.

6 Deadlocks kan uppstå när vissa kriterier är uppfyllda. I ett visst datorsystem råder scenariot som avbildas i resursallokeringsgrafen nedan. Observera att:

- Systemet har ingen preemption eller deadlockförhindrande algoritm.
- I de situationer där flera processer efterfrågar samma resurs samtidigt allokeras resursen slumpartat.
- Processerna kan inte efterfråga fler resurser än de som redan representeras i grafen.
- Samtliga resursinstanser är odelbara.

Besvara följande frågor för det i grafen givna scenariot.



6(a) Har en deadlock uppstått? *Motivera ditt svar*.

Skriv in ditt svar här

Nej, inget deadlock har uppstått. Oberoande av scenariot så kan p4 och p5 släppa sina resurser. Detta ger skansen för p3 att få o sen släpa R3. Då kan p2 och p1 få en instans av R3. Nu finns det två scenario. Om P1 får R0, då kan den släppa R3 och eventuellt R0. Då kan p2 få en instans av R0 och släppa R3 och R1. Då kan p0 entligen få R1. I den andra scenarion så får P2 R0 först. Då Kan den släppa R3 och R1 så att P0 kan få R1. Sen kan P1 få R0. Därför har inget deadlock uppstått eftersom vi är i ett safe lägge.

Ord: 112

Besvarad.

6(b) Resurs R_3 är en dyrbar resurs där varje aktiv instans ökar underhållskostnaderna för vår organisation. Vi önskar därför rationalisera bort så många instanser av R_3 som möjligt. Vi vill dock inte riskera att systemet hamnar i ett osäkert tillstånd (*unsafe state*) där en deadlock skulle kunna uppstå.

Utan att ändra övriga förutsättningar för scenariot, vad är det lägsta antalet instanser av R_3 som krävs för att systemet ska fungera (om än långsamt) samt vara i ett säkert tillstånd (*safe state*) och därmed helt undvika framtida deadlocks?

Välj ett alternativ

- 0
- 1
- 2
- 3
- 4
- 5
- Fler än 5 resursinstanser av R_3 krävs eftersom systemet för nuvarande är i ett unsafe state på grund av för få resursinstanser av R_3.
- Det spelar ingen roll hur många resursinstanser av R_3 vi har. Systemet är ändå i ett unsafe state och riskerar försättas i en deadlock.

Rätt. 3 av 3 poäng.

CPU-schemaläggning används för att bestämma vilken process, eller när stöd finns, vilken tråd som ska exekvera på processorn. Trådar ankommer kontinuerligt till en redokö efter ett event eller avslutad I/O-operation.

I följande scenario, vid tidpunkten t har processorn precis blivit tillgänglig. Två trådar har också precis ankommit till redokön i väntan på att bli schemalagda. Med ytterst avancerat estimerande (en spåkula) kan vi även se att fler trådar snart ankommer till kön, enligt nedan angivna ankomsttider.

Tabell. Redokön vid tidpunkten t.

Köplats (#)	Tråd	Uppskattad CPU-sprint (ms)	Ankomsttid (ms)
1	T_1	10	t
2	T_2	4	t
--	T_3	10	$t+6$
--	T_4	6	$t+10$

Du kan för dessa uppgifter anta att tidskostnaden för context switching är försumbar. Det är enklast att visualisera sig scenariot om du antar att $t = 0$ vid starten.

7(a) Anta att trådarna schemaläggs med kooperativ (icke-spärrande) *Shortest Job First (SJF)*. Vad blir väntetiden för varje tråd och medelväntetiden (tiden varje tråd väntar i snitt)? *Svara exakt*.

T1: 4 ms.

T2: 0 ms.

T3: 14 ms.

T4: 4 ms.

Medelväntetid: 5.5 ms.

Rätt. 5 av 5 poäng.

7(b) Anta att trådarna schemaläggs med *Round Robin (RR)* där tidskvantumet är $q = 4\ ms$. Vad blir väntetiden för varje tråd och medelväntetiden (tiden varje tråd väntar i snitt)? *Svara exakt*.

T1: 18 ms.

T2: 4 ms.

T3: 14 ms.

T4: 14 ms.

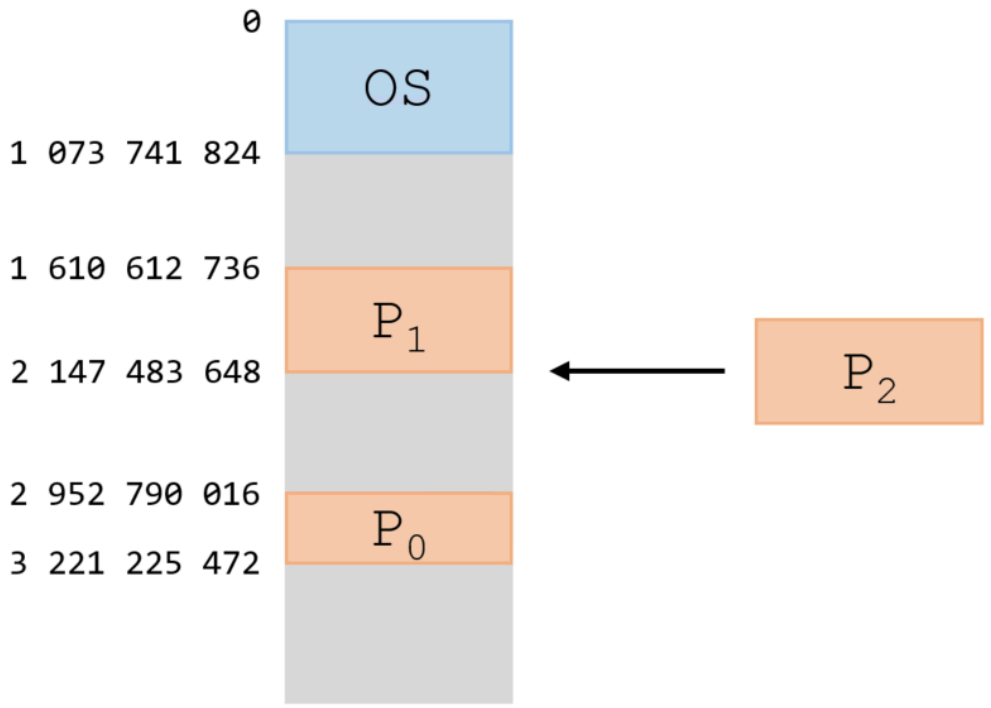
Medelväntetid: 12.5 ms.

Delvis rätt. 2 av 5 poäng.

Sammanhängande allokering kan användas för att allokera processers minne i enhetliga minnesspann.

Anta att du har ett system med 4 GiB primärminne som allokeras sammanhängande. Operativsystemet och två processer har redan allokerats till minnet i detta system, se figuren nedan. I figuren anges adressen för starten av varje block av minne på decimalform. T.ex. är operativsystemet placerat i spannet 0 till och med 1 073 741 823. Observera att figuren inte är skalenlig.

Utrymme behöver nu allokeras för en ny process P_2 som kräver 600 MiB. Besvara följande frågor relaterade till det givna scenariot.



8(a) På vilken adress kommer processens bas placeras om metoden **first-fit** används? *Svara med talbas 10*.

Ange adressen här: 2147483648 .

Rätt. 2 av 2 poäng.

8(b) På vilken adress kommer processens bas placeras om metoden **best-fit** används? *Svara med talbas 10*.

Ange adressen här: 2147483648 .

Rätt. 2 av 2 poäng.

8(c) På vilken adress kommer processens bas placeras om metoden **worst-fit** används? *Svara med talbas 10*.

Ange adressen här: 3221225472 .

Rätt. 2 av 2 poäng.

8(d) Anta nu att du skapar ytterligare processer som alla har samma storlek som P_2. Du upprepar detta tills att det inte längre går att allokera minne till nya sådana processer. Enligt 50-procentregeln, hur mycket minne har nu gått förlorat till extern fragmentering?

Anta att inga processer stängs av och att ingen kompaktering eller swapping sker. Svara i MiB.

Extern fragmentering: 600 MiB.

Fel. 0 av 2 poäng.

8(e) 50-procentregeln är endast en tumregel. Hur mycket minne har egentligen gått förlorat till extern fragmentering?

Extern fragmentering: 1104 MiB.

Rätt. 2 av 2 poäng.

i I ett av dina serverkluster har du enligt loggad statistik de senaste dagarna i snitt haft:

- Ett CPU-nyttjande på 17 %.
- En överföringsbelastning på hårddiskarna där swap finns på 97 %.
- Ett I/O-nyttjande av övriga enheter på 6 %.

Se även följande information från **`top`**:

```
top - 15:16:44 up 98 days, 13:52, 1 user, load average: 4.10, 4.08, 4.11
Tasks: 222 total, 1 running, 221 sleeping, 0 stopped, 7 zombie
```

Klustret exekverar flera CPU-intensiva jobb varför detta är icke-önskvärd statistik. I idealfallet ska CPU-nyttjandet över tid snitta minst 90 %.

9(a) Vilken eller vilka av följande åtgärder kan du tillämpa för att öka CPU-nyttjandet (den procentuella belastningen)?

Välj ett eller flera alternativ:

- [x] Byt ut hårddisken där swap lagras till en solid-state drive (*SSD*).
- [ ] Installera en hårddisk med större lagringskapacitet.
- [ ] Installera en hårddisk med lägre lagringskapacitet.
- [ ] Öka antalet jobb som får startas parallellt.
- [ ] Minska antalet jobb som får startas parallellt.
- [ ] Installera en snabbare CPU.
- [ ] Installera en långsammare CPU.
- [x] Installera mer primärminne.
- [ ] Installera mer sekundärminne.
- [ ] Eliminera init/systemd.
- [ ] Öka maxstorleken på det reserverade swaputrymmet.
- [ ] Ladda ner mer RAM på **`downloadmoreram.com`**.

Observera:

- Markerade åtgärder som bidrar till att öka CPU-nyttjandet ger från noll till flera poäng utifrån effekten.
- Markerade åtgärder som inte bidrar till att öka CPU-nyttjandet ger minuspoäng.
- Omarkerade alternativ ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Delvis rätt. 3 av 6 poäng.

10(a) Anta att du har ett filsystem som använder länkad allokering med 32-bitars adresser och en blockstorlek på 256 B.

Vad blir den maximala filstorleken och overheaden för en fil i detta system? *Svaren anges i GiB*.

Maximal filstorlek: 1024 GiB.

Overhead: 16 GiB.

Rätt. 4 av 4 poäng.

10(b) Anta att du har ett filsystem liknande UFS som använder hierarkisk indexerad allokering, även kallat *combined scheme*, med 32-bitars adresser och en blockstorlek på 256 B. I inoden finns 8 direkta, 1 indirekt och 1 dubbelt indirekt pekare som leder till data- och indexblock.

Vad blir den maximala filstorleken och overheaden i detta system? *Svaren anges i KiB respektive bytes.*

Maximal filstorlek: 1042 KiB.

Overhead: 16936 B.

Rätt. 4 av 4 poäng.

10(c) Du arbetar nu med det tidigare angivna filsystemet med hierarkisk indexerad allokering. Anta att du har möjligheten att ändra adresstorleken. Blockstorleken och övriga egenskaper är oförändrade. Vilken adresstorlek a skulle ge den största möjliga maximala filstorleken?

Du kan anta att adresstorleken a är ett positivt antal bitar: $a \in \mathbb{Z}^+$ som inte nödvändigtvis är en tvåpotens.

Motivera ditt svar.

Skriv in dina beräkningar och ditt svar här

Detta kan ses som en kalkyl fråga där vi vill hitta max punkten på funktionen s(b). Där s är maximala filstorleken och b är adresstorleken. s(b) blir då s(b) = 8 * 256 + (256/(a/8) * 256) + (256 *(256/(a/8)^2). Vi kan förenkla lite genom att tänka i byte direkt istället för bits. Så vi bytter ut ekvationen med s(b) = 8 * 256 + (256/(A) * 256) + (256 *(256/(A)^2) där A är a/8.Då blir derevatan (-256/A^2)(1 + (2*256)/ A). Nu ser vi tydligt att ju större adresstorleken är ju mindre av dom vi får plats i ett block och därför blir maxstorleken mindre. Men svaret är såklart inte att min vill ha en 1 bit stor adress, då får vi inte plats med någonting alls. Vi måste kunna adressera varje block vi använder. Nu vet vi att vi helt enkelt vill ha minsta möjliga värdet på a som uppfyller 2^A = 256/A + 8 + (256/A)^2. Jag kan inte beräkna detta för hand.

Ord: 165

Besvarad.

Du har nu nått slutet av tentamen. Bra jobbat!

Inte klar med tentan än? Gå tillbaks och kör hårt! Kom tillbaks hit när du känner dig nöjd.

Om du inte redan fått en länk till kursvärderingen så kommer den inom kort till din e-post. Det är av stort värde för mig att du fyller i den så gör gärna det.

Slutligen en kontrollfråga som inte påverkar bedömningen ...

Om du installerar tjugo SD-kort om 128 GB vardera på en brevduva och ber den flyga till Stockholm och tillbaks, vilken effektiv överföringshastighet får du då?

- Med maskhål kan vi bryta Minkowskirummet och få ett avstånd *a* som närmar sig noll vilket ger en närmast oändlig överföringshastighet $\lim_{a \to 0^+} \frac{x}{a} = \infty$
- Det beror främst på vår packet loss (tappade SD-kort)
- Alla vet att brevugglor hade varit ett bättre alternativ
- Cirka 250 Mbit/s

Tack för den här kursomgången! Vi ses.

Vänligen,
Jimmy

Besvarad.